Sistema de Gestión de Estudiantes (SGE) básico

Joan Sebastián Quiñones Arias

Fundación Universitaria Compensar

Algoritmos y Programación, grupo 2310 - 1A

Eliana Marcela Paez Perdomo

2 de noviembre de 2026

Sistema de Gestión de Estudiantes (SGE) básico

En cualquier institución educativa, por pequeña que sea, hay una tarea que se repite todo el tiempo: registrar estudiantes, consultar quiénes están inscritos y encontrar rápidamente la información de una persona en particular. Cuando esto se hace en papel o en hojas sueltas aparecen problemas conocidos, como datos duplicados, registros incompletos y mucho tiempo perdido buscando un nombre. Este documento presenta el diseño, la codificación y las pruebas de un programa que resuelve ese problema de forma básica y que, al mismo tiempo, sirve para aplicar los fundamentos de algoritmos vistos en la asignatura.

Contexto y objetivo del sistema

El objetivo del trabajo fue diseñar, codificar y probar un Sistema de Gestión de Estudiantes (SGE) básico que funciona por consola y que permite agregar estudiantes, mostrarlos, buscarlos por su identificador (ID) y salir del sistema. Más allá del programa en sí, el propósito académico es demostrar el uso consciente de las estructuras de control, de las variables especiales (contadores, banderas y acumuladores) y de una estructura de datos simple, que según Joyanes Aguilar (2008) son los elementos con los que se construye cualquier algoritmo. También se analiza qué tan eficiente es la solución usando la notación Big-O.

Problema que resuelve el software

El SGE resuelve el problema de mantener, durante una sesión de trabajo, un registro ordenado y confiable de estudiantes. En concreto, el sistema:

- evita IDs duplicados, porque antes de guardar un estudiante verifica que su identificador no exista;
- evita datos inválidos (edades imposibles, notas fuera de la escala de 0.0 a 5.0, nombres con números), porque valida cada entrada y vuelve a pedirla si está mal;
- permite consultar la lista completa en dos órdenes (de registro e inverso) y buscar a una persona por su ID, informando si se encontró o no;
- controla que no se supere la capacidad máxima definida, como ocurriría con un arreglo de tamaño fijo, y

- no se cierra con un error si el usuario escribe texto donde se espera un número, ni si interrumpe el programa con `Ctrl+C`.

Tecnologías y lenguaje de programación utilizados

El programa se escribió en Python, que fue el lenguaje asignado para la asignatura. Se necesita la versión 3.10 o superior porque se usa la sentencia `match-case`, introducida en esa versión como mecanismo de coincidencia de patrones y que cumple el papel del `switch` de otros lenguajes (Bucher y van Rossum, 2020). En la Tabla 1 se resumen todas las herramientas empleadas.

Tabla 1

Herramientas utilizadas en el proyecto

Herramienta	**Uso en el proyecto**
Python 3.13 (compatible desde 3.10)	Lenguaje de programación del sistema.
Biblioteca estándar de Python	Se usaron solo `input()`, `print()`, listas, diccionarios y manejo de excepciones, sin librerías externas, para que el programa corra en cualquier computador con Python instalado.
Mermaid	Elaboración de los diagramas de flujo a partir de texto (Mermaid, s. f.).
Git y GitHub	Control de versiones y publicación del código fuente.
Terminal o consola	Ejecución del programa y obtención de las evidencias.

Descripción general de la solución

El programa se organiza en un ciclo principal que muestra un menú con cinco opciones mientras la bandera `bandera_salir` sea falsa. Según la opción elegida, una estructura de decisión múltiple (`match-case`) llama a la función correspondiente. Los estudiantes se guardan en una lista de diccionarios, en la que cada diccionario representa a un estudiante con los campos *id*, *nombre*, *edad*, *programa* y *nota*. La variable `total_estudiantes` funciona como contador y se compara con la constante `MAX_ESTUDIANTES = 50` para no exceder la capacidad.

Además de las cuatro funcionalidades obligatorias, agregué una opción de estadísticas (promedio general, número de aprobados y mejor nota). La incluí porque permite demostrar el

uso de un acumulador, que es otra variable especial, y un recorrido completo de la lista en una sola pasada.

Planificación y desarrollo iterativo

No programé todo de una vez. Primero hice el diagrama de flujo y lo validé con una prueba de escritorio, y luego desarrollé por funcionalidades. Al final de cada iteración ejecutaba el programa para comprobar que lo nuevo funcionaba sin dañar lo anterior. La Tabla 2 muestra el orden que seguí.

Tabla 2

Iteraciones del desarrollo

Iteración	Qué se construyó	Cómo se comprobó
1	Menú principal con el ciclo `while`, la bandera de salida y el `match-case` (las opciones solo imprimían un mensaje).	Navegar el menú, escribir opciones inválidas y salir con la opción 5.
2	Estructura de datos (lista vacía, contador y constante de capacidad) y funciones de lectura validada.	Escribir letras, números negativos y valores fuera de rango.
3	Función Agregar, con control de ID repetido y de capacidad.	Registrar estudiantes, repetir un ID y bajar la capacidad a 2.
4	Función Mostrar, con recorrido ascendente y descendente.	Listar cuatro estudiantes en ambos órdenes.
5	Función Buscar (búsqueda lineal con bandera y contador de comparaciones).	Buscar el primero, uno intermedio y un ID inexistente.
6	Estadísticas, manejo de `Ctrl+C` y ajustes de formato.	Ejecución de todos los casos de prueba.

Diagramas de flujo

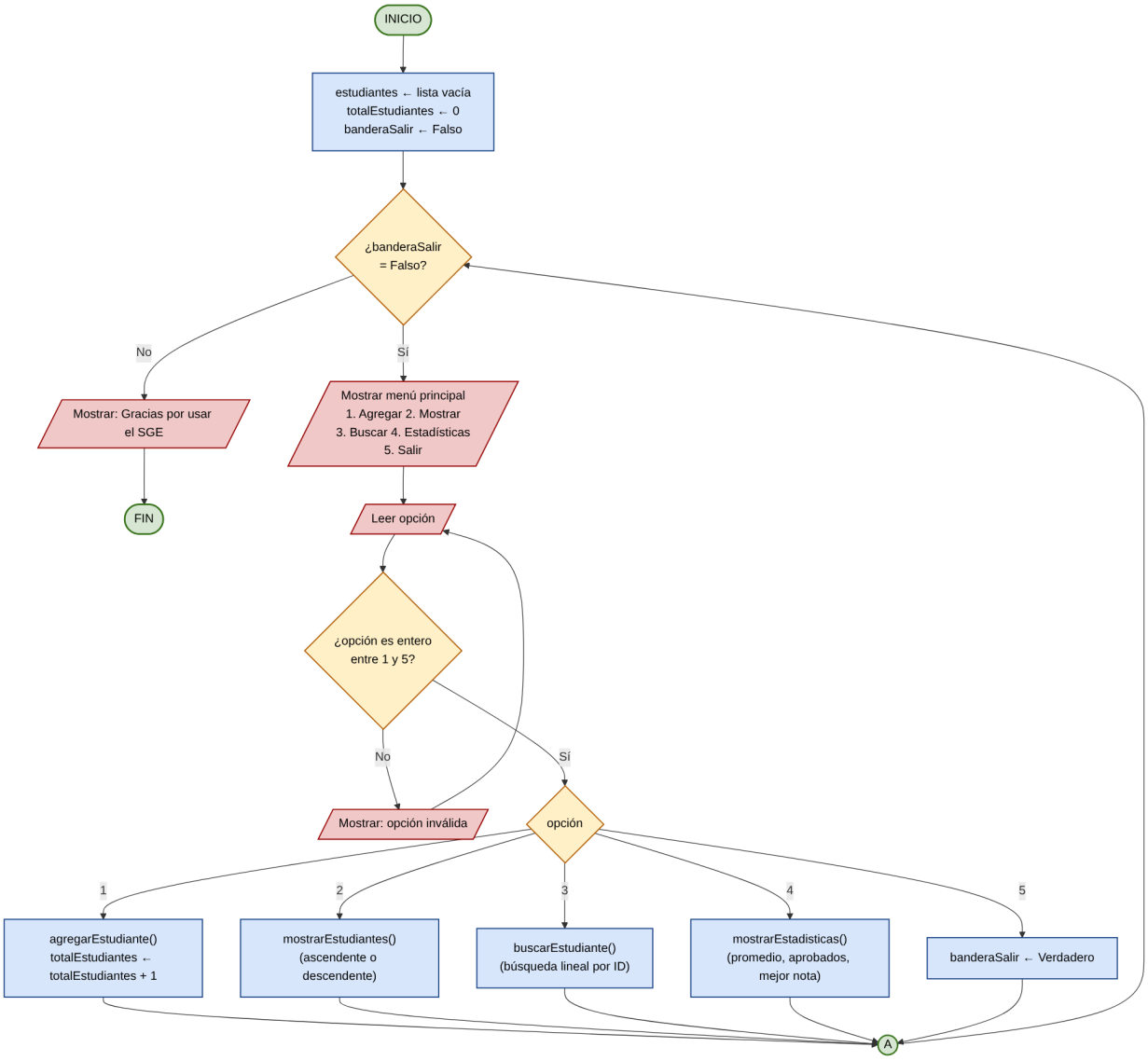
Antes de escribir código elaboré el diagrama del algoritmo principal y, como algunas opciones tienen su propia lógica interna, también los diagramas de los procedimientos más importantes. Usé la simbología estándar definida en la norma ISO 5807 (International Organization for Standardization [ISO], 1985) y, para facilitar la lectura, un color distinto para cada tipo de símbolo.

Diagrama de flujo principal

La Figura 1 representa el algoritmo completo del sistema: la inicialización de las variables, el ciclo controlado por la bandera, la validación de la opción, la decisión múltiple que dirige el flujo a cada funcionalidad y el regreso al menú a través del conector A.

Figura 1

Diagrama de flujo principal del SGE

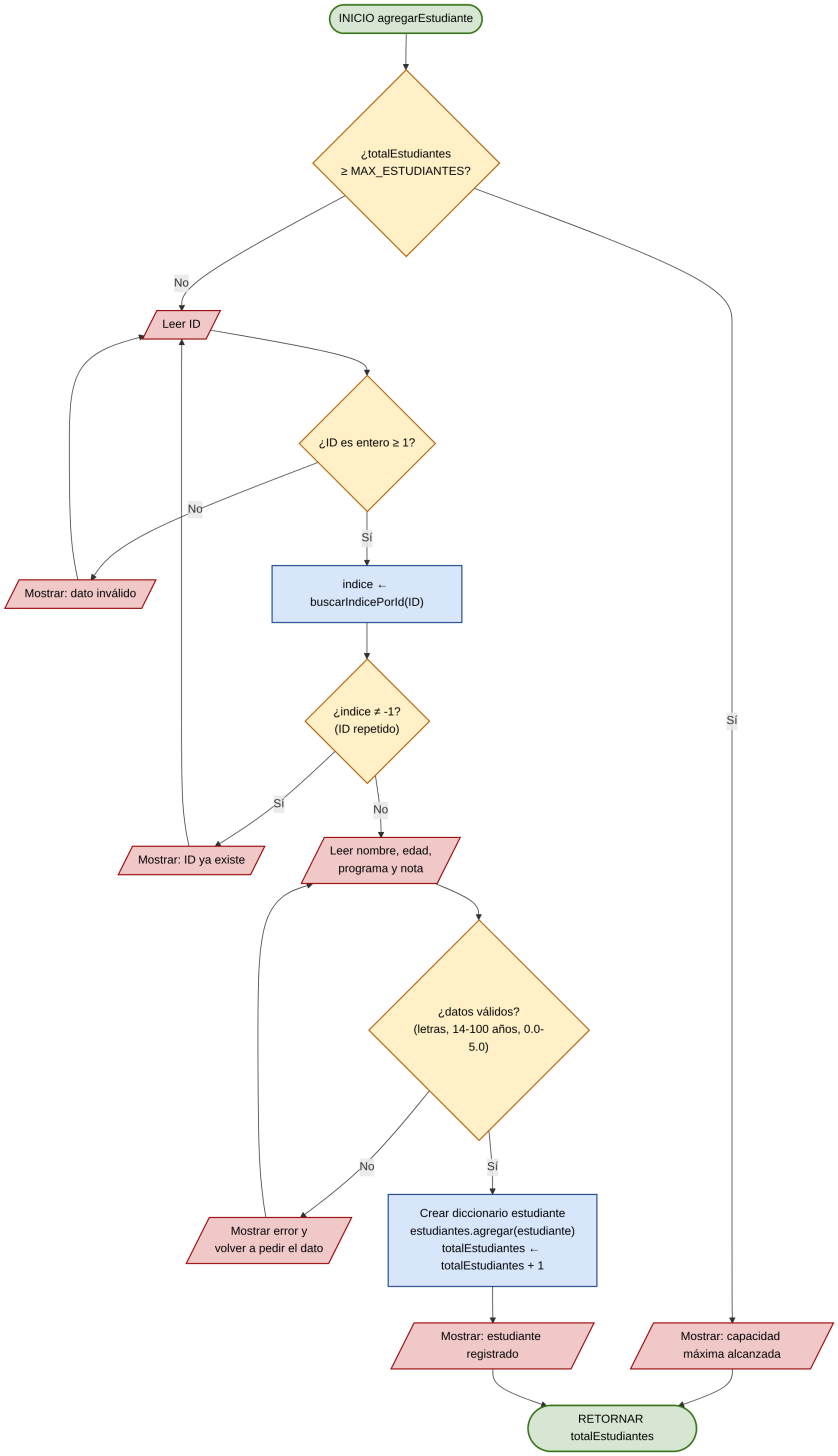


Procedimiento Agregar estudiante

La Figura 2 detalla el procedimiento de registro. Primero verifica la capacidad, después valida que el ID sea un entero positivo y que no esté repetido, y por último pide y valida el resto de los datos antes de guardarlos y aumentar el contador.

Figura 2

Diagrama de flujo del procedimiento agregarEstudiante

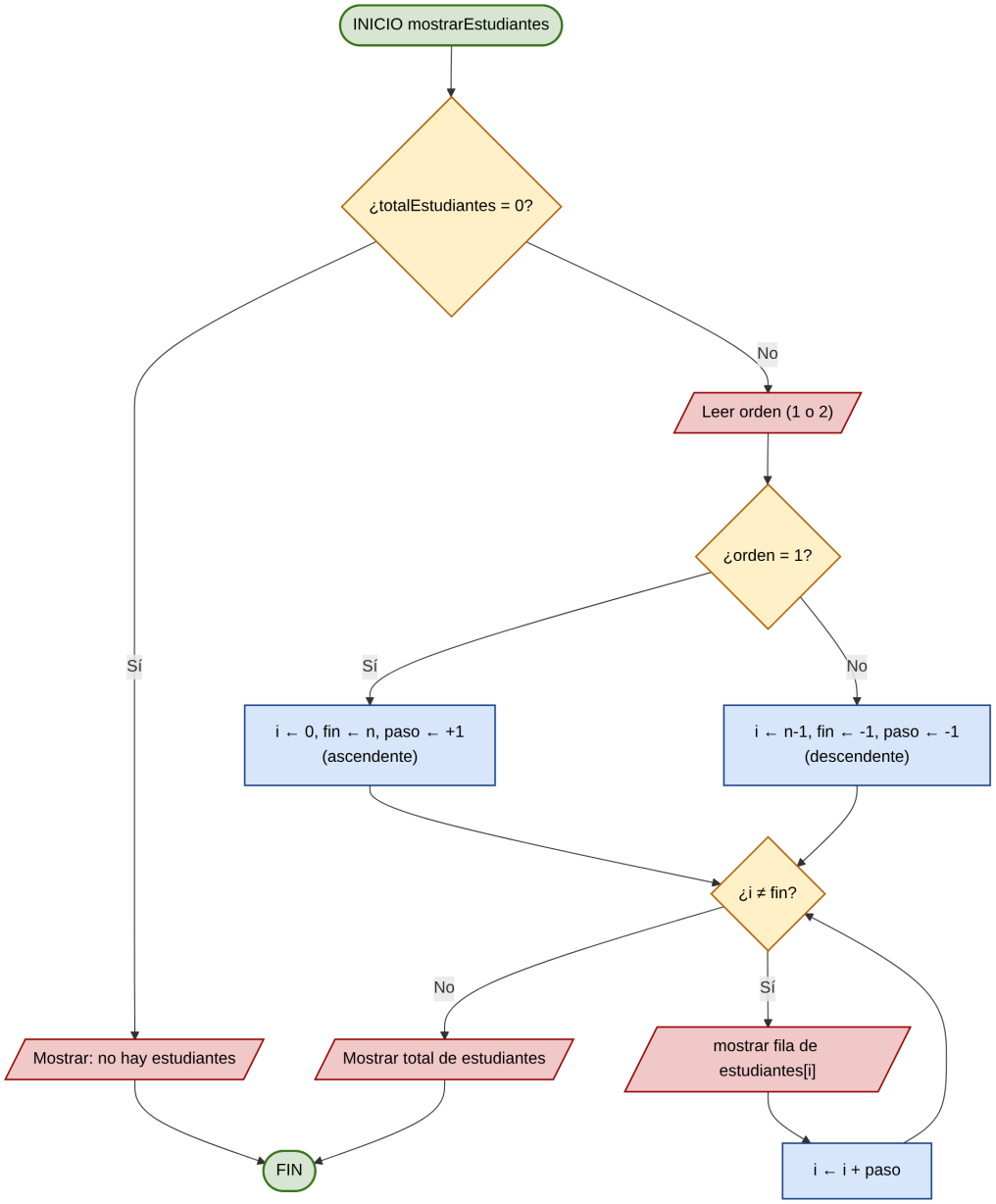


Procedimiento Mostrar estudiantes

La Figura 3 muestra cómo un mismo ciclo sirve para recorrer la lista de forma ascendente (paso +1) o descendente (paso −1), según lo que elija el usuario.

Figura 3

Diagrama de flujo del procedimiento mostrarEstudiantes

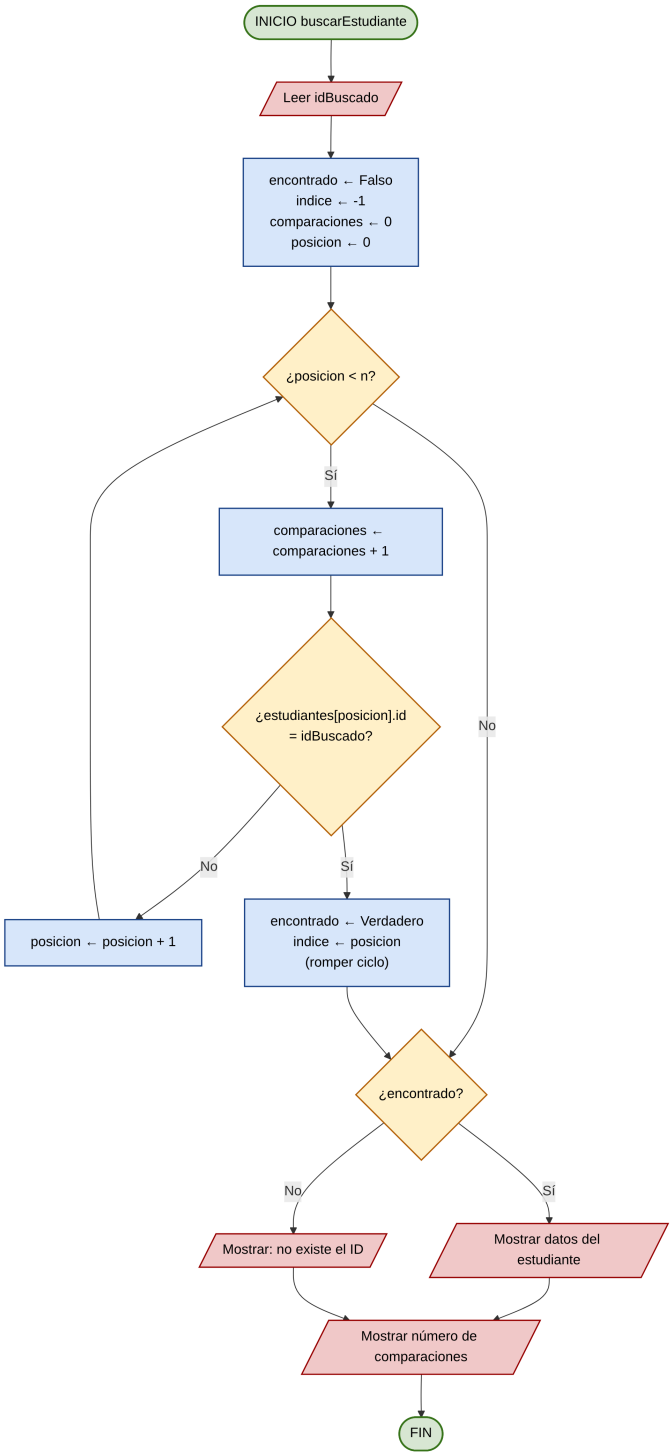


Procedimiento Buscar estudiante por ID

La Figura 4 representa la búsqueda lineal. Se usa la bandera *encontrado* y el ciclo termina en cuanto aparece el ID buscado o se acaba la lista.

Figura 4

Diagrama de flujo de la búsqueda lineal por ID



Justificación de los símbolos utilizados

Cada símbolo del diagrama tiene un significado definido por la norma ISO 5807 (ISO, 1985) y una correspondencia directa con una instrucción de Python. En la Tabla 3 se explican los símbolos empleados.

Tabla 3

Símbolos del diagrama de flujo y su correspondencia en Python

Símbolo	Nombre	Uso en el diagrama	Correspondencia en Python
	Óvalo (terminal)	Marca dónde empieza y termina el algoritmo o un procedimiento (INICIO, FIN, RETORNAR).	Inicio de `main()` o de una función (`def`), y su `return` o final.
	Rectángulo (proceso)	Asignaciones y cálculos: inicializar variables, incrementar el contador, activar la bandera, agregar a la lista o llamar a un procedimiento.	`total_estudiantes += 1`, `bandera_salir = True`, `append()`.
	Rombo (decisión)	Una pregunta cuya respuesta cambia el camino. Con dos salidas es una decisión simple o doble; con varias (1 a 5) es una decisión múltiple. Si una salida regresa a un punto anterior, forma un ciclo.	`if`, `elif`, `else`, `match-case` y la condición de `while` o `for`.
	Paralelogramo (entrada y salida)	Datos que entran desde el teclado (leer opción, leer ID) o que se muestran en pantalla (menú, mensajes, listados).	`input()` y `print()`.
	Círculo (conector)	Reúne en un punto las salidas de todas las opciones del menú para regresar al ciclo principal sin cruzar tantas flechas.	El final de cada `case`, después del cual se vuelve a evaluar el `while`.
	Flecha (línea de flujo)	Indica el orden de ejecución. Las flechas que suben representan la repetición.	El orden secuencial de las instrucciones y el regreso de los ciclos.

Validación del diagrama con una prueba de escritorio

Antes de codificar, recorrí el diagrama de búsqueda a mano con una lista de ejemplo de cuatro IDs (1001, 1002, 1003 y 1004), buscando el ID 1003. Como se observa en la Tabla 4, el ciclo se detiene apenas encuentra el ID y el contador de comparaciones queda en 3. El programa real dio después el mismo resultado (ver Figura 11).

Tabla 4

Prueba de escritorio de la búsqueda del ID 1003

Iteración	posicion	¿posicion < n?	ID en la posición	¿Es 1003?	comparaciones	encontrado
1	0	Sí	1001	No	1	Falso
2	1	Sí	1002	No	2	Falso
3	2	Sí	1003	Sí	3	Verdadero

Nota. En la iteración 3 la bandera cambia a verdadero y el ciclo termina sin revisar la posición 3.

También hice la prueba de escritorio del ciclo principal con la secuencia de opciones 1, 2 y 5. La bandera inicia en falso y el ciclo se repite dos veces (agregar y mostrar). En la tercera vuelta, la opción 5 cambia la bandera a verdadero, la condición “¿banderaSalir = Falso?” da “No” y el algoritmo llega a FIN. Con esto confirmé que el ciclo no es infinito y que siempre existe una salida.

Análisis de estructuras de control y variables especiales

Las estructuras de control determinan el orden en que se ejecutan las instrucciones de un programa. Python ofrece la decisión (`if`), la repetición (`while` y `for`), la interrupción de ciclos (`break`) y, desde la versión 3.10, la decisión múltiple con `match` (Python Software Foundation [PSF], s. f.-b). Para los errores en tiempo de ejecución se dispone de las sentencias `try` y `except` (PSF, s. f.-a).

Identificación de estructuras de control

La Tabla 5 identifica cada estructura de control usada en `sge.py`, dónde está y por qué la elegí para ese escenario. Los números de línea corresponden al listado del Apéndice A.

Tabla 5

Estructuras de control utilizadas y justificación de su uso

Estructura	Tipo	Ubicación	Escenario y justificación
`while not bandera_salir:`	Repetición condicionada al comienzo	`main()`, línea 284	Se usa para repetir el menú principal hasta que el usuario decida salir. Como no se sabe cuántas veces usará el sistema, no tiene sentido un `for` con un número fijo de vueltas.
`match opcion:` con `case 1` a `case 5`	Decisión múltiple (switch)	`main()`, líneas 289–299	Se emplea para dirigir el flujo a la opción específica (agregar, mostrar, buscar, estadísticas o salir). Con cinco opciones es más legible que una cadena de `elif`.
`while True:` con `return`	Repetición con salida interna (simula un do-while)	Funciones `leer_*`, líneas 44, 62 y 77	Un dato se debe pedir al menos una vez y repetirse mientras sea inválido. Python no tiene `do-while`, así que lo simulé con `while True` y un `return` cuando el dato es válido.
`try` y `except ValueError`	Manejo de excepciones	Líneas 46–50 y 65–69	Anticipa que el usuario escriba texto donde se espera un número; `int("doce")` lanzaría `ValueError` y cerraría el programa.
`try` y `except (KeyboardInterrupt, EOFError)`	Manejo de excepciones	Líneas 305–308	Si el usuario presiona `Ctrl+C`, el programa termina de forma ordenada, sin mostrar un error técnico.
`if`, `elif` y `else`	Decisión doble anidada	`leer_entero` (52–56) y `leer_texto` (79–86)	Verifica varias condiciones en orden (menor que el mínimo, mayor que el máximo o válido). Solo se ejecuta una rama.

Estructura	Tipo	Ubicación	Escenario y justificación
`if total_estudiantes >= MAX_ESTUDIANTES:`	Decisión simple	`agregar_estudiante`, línea 154	Controla que no se exceda el tamaño máximo. Si la lista está llena, se informa y se sale de la función sin pedir datos.
`while indice != -1:`	Repetición condicionada al comienzo	`agregar_estudiante`, línea 161	Mientras el ID ya exista, se pide otro. Si el ID es nuevo desde el principio, el ciclo no se ejecuta ninguna vez.
`for posicion in range(len(estudiantes)):` con `break`	Repetición ascendente con salida anticipada	`buscar_indice_por_id`, líneas 108–113	Recorre la lista desde la posición 0. El `break` corta el ciclo apenas se encuentra el ID y ahorra comparaciones.
`if estudiantes[posicion]["id"] == id_buscado:`	Decisión simple	Línea 110	Compara cada elemento con el ID buscado y, si coincide, activa la bandera `encontrado`.
`if indice != -1:` con `else`	Decisión doble	`buscar_estudiante`, línea 227	Se implementa para validar si un estudiante fue encontrado o no durante la búsqueda y mostrar el mensaje que corresponde.
`for posicion in range(inicio, fin, paso):`	Repetición ascendente o descendente	`mostrar_estudiantes`, línea 208	Un solo `for` sirve para los dos órdenes: `range(0, n, 1)` avanza y `range(n-1, -1, -1)` retrocede. Los parámetros se eligen con un `if` en la línea 198.
`for estudiante in estudiantes:`	Repetición sobre una colección	`mostrar_estadisticas`, línea 248	Cuando no se necesita la posición sino cada elemento, este `for` es más claro. En una sola pasada acumula la suma, cuenta los aprobados y encuentra la mejor nota.
`"Aprobado" if … else "Reprobado"`	Decisión doble en una expresión	`mostrar_estudiante`, línea 121	Elige un texto según la nota en una sola línea, sin abrir un bloque completo para algo tan corto.

Variables especiales

Joyanes Aguilar (2008) distingue entre las variables que almacenan datos y las que cumplen un papel de control dentro del algoritmo, como los contadores, los acumuladores y los interruptores o banderas. La Tabla 6 describe las variables especiales del SGE y la función que cumple cada una.

Tabla 6

Variables especiales del programa

Variable	Tipo	Función estratégica en el programa
`total_estudiantes`	Contador	Lleva el total de estudiantes registrados y aumenta en 1 con cada registro (línea 180). Controla la capacidad máxima, indica si la lista está vacía y sirve para calcular el promedio. La mantuve como variable aparte, aunque `len()` daría el mismo valor, para que el patrón de contador quede explícito y se pueda mostrar en el menú (por ejemplo, [4/50]).
`bandera_salir`	Bandera	Controla la salida del ciclo principal. Empieza en `False` y solo la opción 5 la cambia a `True`, así que el `while` tiene una única condición de terminación.
`encontrado`	Bandera	Indica dentro de la búsqueda si el ID ya apareció; su valor determina lo que retorna la función.
`comparaciones`	Contador	Cuenta cuántas comparaciones hizo la búsqueda. Se muestra al usuario como evidencia práctica del análisis Big-O.
`suma_notas`	Acumulador	Suma las notas de todos los estudiantes para calcular el promedio general.
`aprobados`	Contador condicional	Solo aumenta cuando la nota es mayor o igual a 3.0.
`mejor`	Variable auxiliar de máximo	Guarda el estudiante con la nota más alta encontrado hasta el momento.
`MAX_ESTUDIANTES`, `NOTA_APROBATORIA` y otras	Constantes	Centralizan los valores de configuración; para cambiar el tope o la nota mínima basta con modificar una línea.

Justificación de la estructura de datos elegida

Elegí una lista de diccionarios. La lista mantiene el orden de registro, que se necesita para mostrar en orden ascendente y descendente, y permite agregar elementos al final con `append()` en tiempo constante amortizado (PSF, s. f.-c). Cada estudiante es un diccionario porque así sus

datos se acceden por nombre (`estudiante["nota"]`) y no por una posición difícil de recordar, como sería `estudiante[4]` si se usaran tuplas.

Aunque las listas de Python crecen solas, definí un tamaño máximo (`MAX_ESTUDIANTES = 50`) y lo controlo con el contador. Lo hice para practicar el control de límites que exige un arreglo de tamaño fijo en lenguajes como C o Java. Este límite se comprobó en el caso de prueba CP-13.

Análisis de eficiencia algorítmica

La notación Big-O describe una cota superior para el crecimiento del tiempo de ejecución de un algoritmo cuando aumenta el tamaño de la entrada, *n* (Cormen et al., 2022). En el SGE, *n* es el número de estudiantes registrados. En la Tabla 7 se analiza cada operación en su mejor caso, caso promedio y peor caso.

Tabla 7

Complejidad temporal de las operaciones del SGE

Operación	Mejor	Promedio	Peor	Explicación
`buscar_indice_por_id`	O(1)	O(n)	O(n)	Si el ID es el primero basta una comparación. En promedio se revisa la mitad de la lista (n/2) y, como se ignoran las constantes, sigue siendo O(n). Si el ID no existe se revisan los *n* elementos.
`agregar_estudiante`	O(1)[a]	O(n)	O(n)	`append()` es O(1) amortizado, pero antes hay que verificar que el ID no esté repetido mediante una búsqueda lineal.
`mostrar_estudiantes`	O(1)[b]	O(n)	O(n)	Se imprime cada estudiante una vez. Ambos órdenes cuestan lo mismo, porque el `range` descendente no copia ni invierte la lista.
`mostrar_estadisticas`	O(1)[b]	O(n)	O(n)	Un único recorrido calcula suma, aprobados y máximo. Tres recorridos separados también serían O(n), pero con el triple de trabajo (3n).
Funciones `leer_*` y `mostrar_estudiante`	O(1)	O(1)	O(1)	Trabajan sobre un solo dato y no dependen de *n*.
Llenar el sistema con *n* registros		$O(n^2)$		La verificación de duplicados suma 0 + 1 + … + (n − 1) = n(n − 1)/2 comparaciones.
Memoria		O(n)		Un diccionario por estudiante; contadores y banderas ocupan espacio constante.

Nota. [a]Cuando la lista está llena y la función termina de inmediato. [b]Cuando no hay estudiantes registrados.

Comprobación práctica

Para no quedarme solo con la teoría, el programa muestra el contador de comparaciones en cada búsqueda. Con cuatro estudiantes se obtuvo 1 comparación al buscar el primero (mejor caso, Figura 12), 3 al buscar el tercero (Figura 11) y 4 al buscar un ID inexistente (peor caso, Figura 13), lo que coincide con el análisis.

Además, escribí un script de medición (`pruebas/medir_eficiencia.py`) que compara el tiempo del peor caso de la búsqueda lineal con una búsqueda en diccionario para distintos tamaños de *n*. Los resultados aparecen en la Tabla 8.

Tabla 8

Tiempo promedio de búsqueda en el peor caso según el número de estudiantes

n (estudiantes)	Búsqueda lineal (µs)	Diccionario (µs)
50	1.47	0.040
1 000	28.16	0.040
10 000	319.91	0.055
100 000	2 788.59	0.045

Nota. Promedio de 200 repeticiones en el entorno de pruebas. Los valores absolutos cambian según el computador; lo relevante es la tendencia. µs = microsegundos.

Los datos confirman el crecimiento lineal: cuando *n* se multiplica por 10, el tiempo de la búsqueda lineal también se multiplica aproximadamente por 10. El diccionario, en cambio, se mantiene casi constante, porque su búsqueda es O(1) en el caso promedio gracias a la tabla hash (PSF, s. f.-c).

Optimizaciones aplicadas y factores considerados

- **Salida anticipada con `break`.** La búsqueda termina en cuanto encuentra el ID. No cambia la clase O(n), pero en promedio reduce el trabajo a la mitad.
- **Un solo recorrido para las estadísticas.** Se calculan tres resultados en una pasada (n operaciones en lugar de 3n).
- **Reutilización de la búsqueda.** La misma función sirve para buscar y para validar duplicados, en vez de escribir dos recorridos.
- **Recorrido descendente sin copiar la lista.** `range(n-1, -1, -1)` evita crear una lista invertida, que costaría O(n) de memoria adicional.
- **Validación temprana.** La capacidad se revisa antes de pedir los datos, para que el usuario no llene un formulario que sería rechazado.

Consideré guardar los estudiantes en un diccionario `{id: estudiante}`, con el cual buscar y validar duplicados sería O(1). Lo descarté por tres razones. Primero, la actividad pide trabajar con un arreglo o lista. Segundo, con la capacidad de 50 estudiantes el peor caso tarda alrededor de 1.5 microsegundos, una diferencia imperceptible. Tercero, la lista conserva el orden

de registro que se necesita para los listados. Si el sistema tuviera que manejar miles de estudiantes, la mejora lógica sería usar ambos: la lista para el orden y un diccionario como índice por ID.

También descarté la búsqueda binaria, que es O(log n) pero exige que los datos estén ordenados (Bhargava, 2016). Los IDs llegan en cualquier orden, así que habría que insertar cada estudiante en su posición correcta, lo que obliga a desplazar elementos (O(n) por inserción), y además se perdería el orden de registro.

Código fuente

El código completo está publicado en GitHub en https://github.com/Sebastian20031221/Sistema_de_gestion_de_estudiantes-SGE- y su listado, con números de línea, se presenta en el Apéndice A. El repositorio contiene el programa principal (`sge.py`), un archivo `README.md` con las instrucciones de ejecución, los diagramas de flujo y la carpeta `pruebas` con los casos usados para las evidencias. Para ejecutarlo solo se necesita Python 3.10 o superior y el comando `python sge.py`.

Organización en funciones

Separé las responsabilidades para que cada función haga una sola tarea. Esto facilita leer el código, probar cada parte y reutilizarla. La Tabla 9 resume las funciones del programa.

Tabla 9

Funciones del programa y su reutilización

Función	Responsabilidad	Se reutiliza en
`leer_entero`	Pide un entero válido dentro de un rango.	Menú, ID, edad, orden y búsqueda
`leer_decimal`	Pide un decimal y acepta coma o punto.	Nota
`leer_texto`	Pide un texto no vacío y le da formato.	Nombre y programa
`buscar_indice_por_id`	Búsqueda lineal; retorna la posición (o −1) y el número de comparaciones.	Agregar y Buscar
`mostrar_estudiante`	Imprime la ficha de un estudiante.	Buscar
`imprimir_fila` e `imprimir_titulo`	Dan un formato uniforme a la consola.	Todas las opciones
`agregar_estudiante`, `mostrar_estudiantes`, `buscar_estudiante` y `mostrar_estadisticas`	Una función por opción del menú.	`main`
`mostrar_menu` y `main`	Ciclo principal y despacho de opciones.	Punto de entrada

Características del código

Los nombres de variables y funciones son descriptivos (`bandera_salir` en lugar de `flag1`), siguiendo la convención de Python de minúsculas con guion bajo. Los comentarios explican el porqué de las decisiones y no solo lo que hace cada línea; por ejemplo, junto al tope de 50 se aclara que simula un arreglo de tamaño fijo. Todas las entradas del usuario pasan por funciones de validación con manejo de excepciones, de modo que ningún dato incorrecto detiene el programa.

Evidencias de ejecución y casos de prueba

Ejecuté el programa en una terminal con Python 3.13. Para que las pruebas se pudieran repetir siempre de la misma forma, guardé las entradas de cada caso en archivos de texto dentro de la carpeta `pruebas`. En las capturas, lo que escribe el usuario aparece resaltado en blanco, los mensajes de error en amarillo o rojo y las confirmaciones en verde.

Menú principal y validación de opciones

Al iniciar, el menú muestra el contador [0/50]. Como se ve en la Figura 5, al escribir texto (abc) o un número fuera de rango (9) el programa muestra un aviso y vuelve a pedir la opción sin cerrarse. Luego se eligió Mostrar con la lista vacía; este es el estado antes de agregar estudiantes. La Figura 6 muestra el intento de búsqueda sin datos.

Figura 5

Menú principal, opciones inválidas y lista vacía

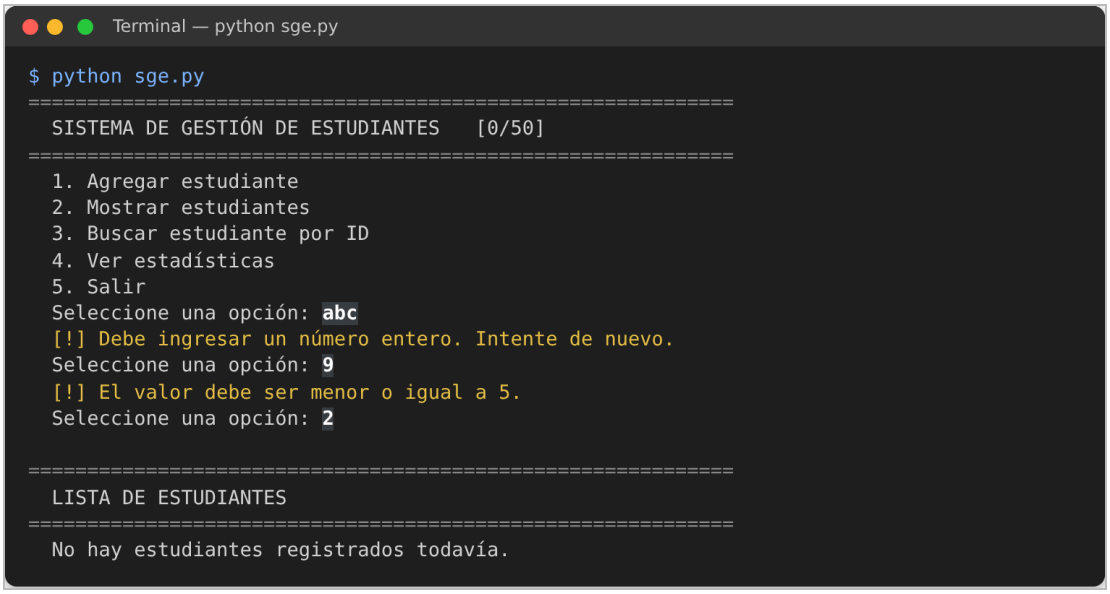
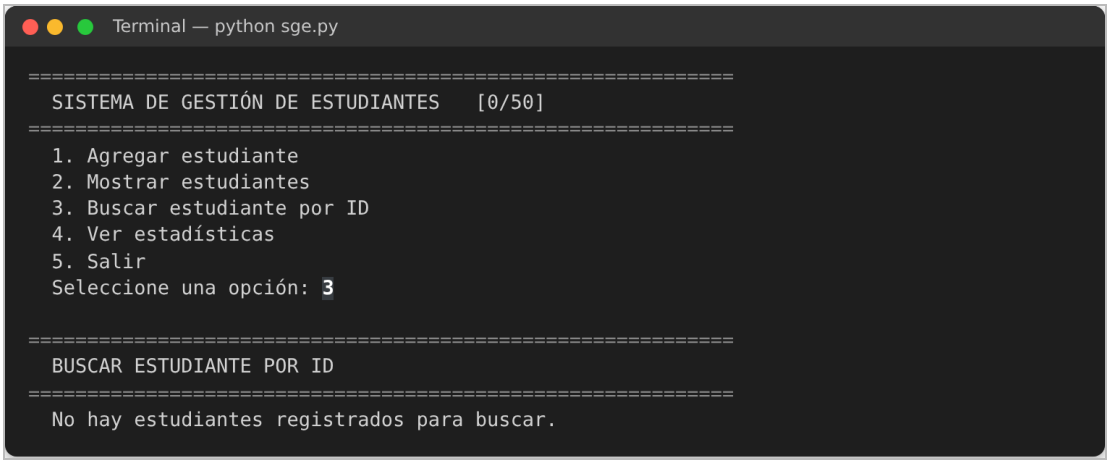


Figura 6

Búsqueda cuando no hay estudiantes registrados

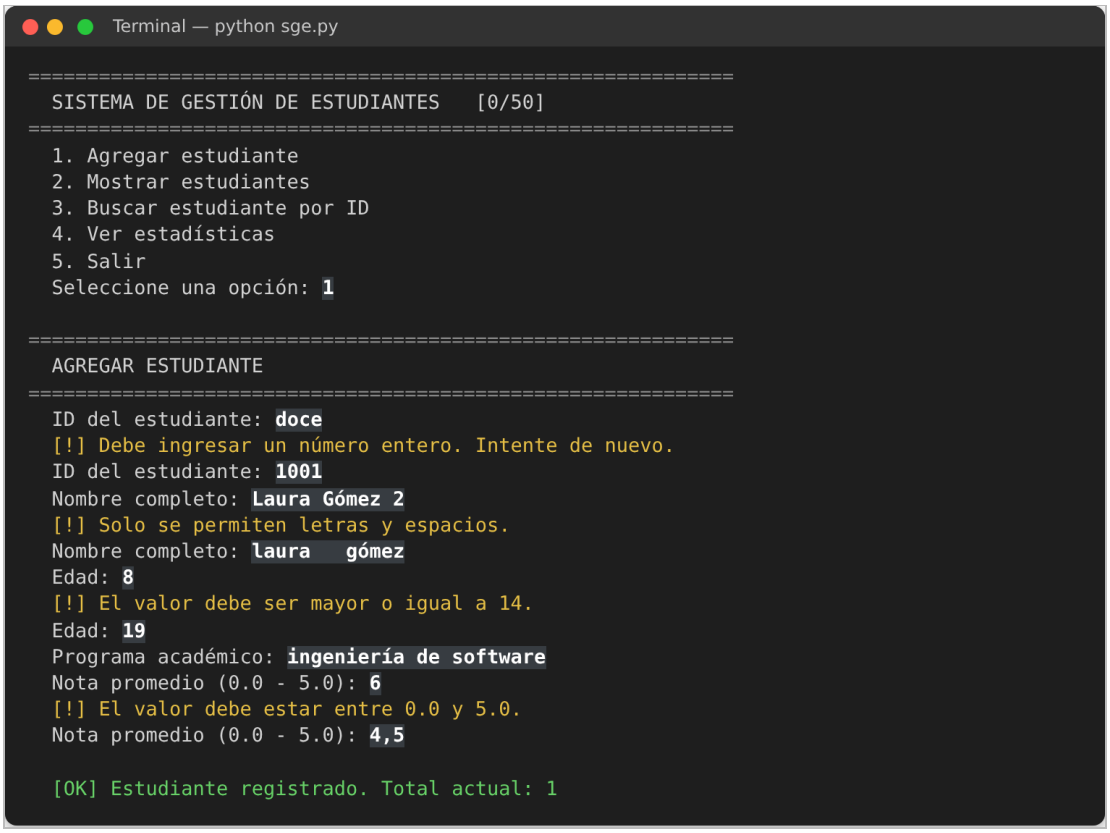


Agregar estudiante

En la Figura 7 se registra el primer estudiante introduciendo errores intencionales en cada campo, y en la Figura 8 se intenta registrar un ID que ya existe.

Figura 7

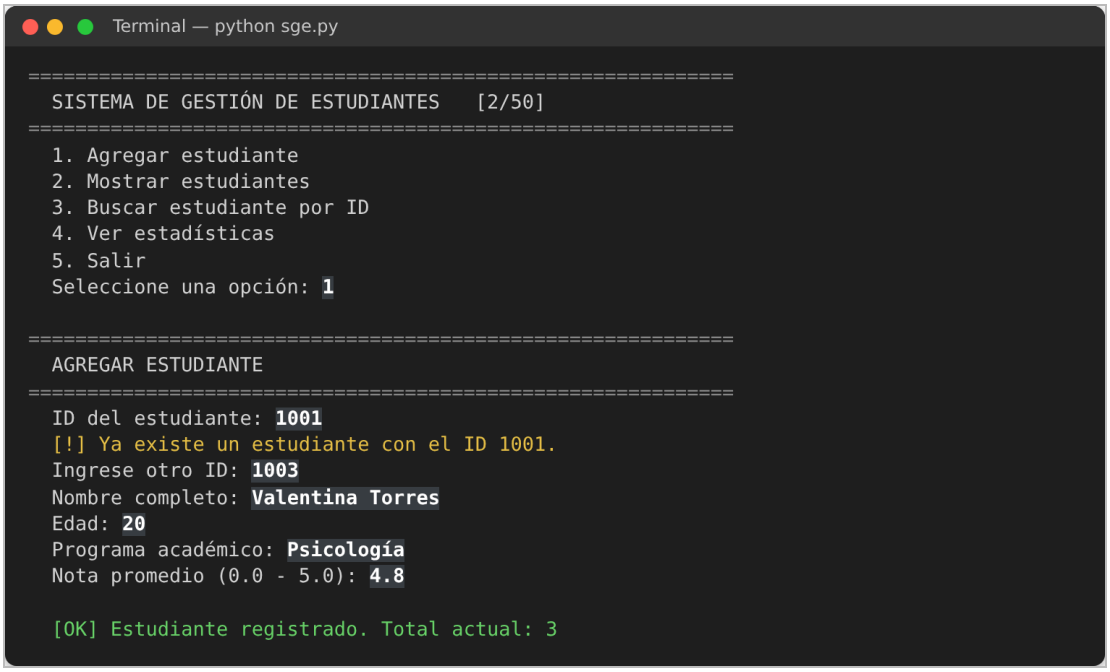
Registro del primer estudiante con validación de cada campo



Nota. Se rechazan un ID con letras, un nombre con un número, una edad menor a 14 y una nota mayor a 5.0. La nota escrita con coma (4,5) se acepta.

Figura 8

Intento de registrar un ID repetido



Mostrar lista con múltiples registros

Después de agregar cuatro estudiantes, el contador del menú pasa a [4/50] y la lista muestra todos los registros. Al comparar las Figuras 9 y 10 con la Figura 5 se aprecia el estado antes y después de la operación de agregar.

Figura 9

Listado en orden de registro (iteración ascendente)

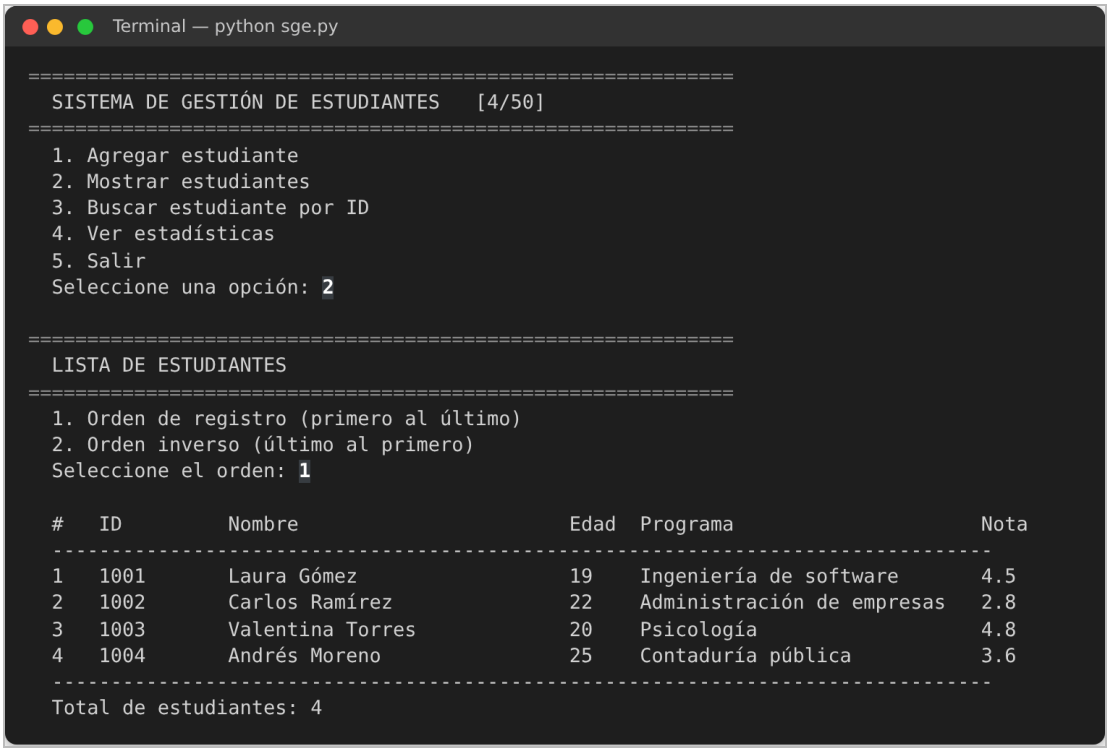
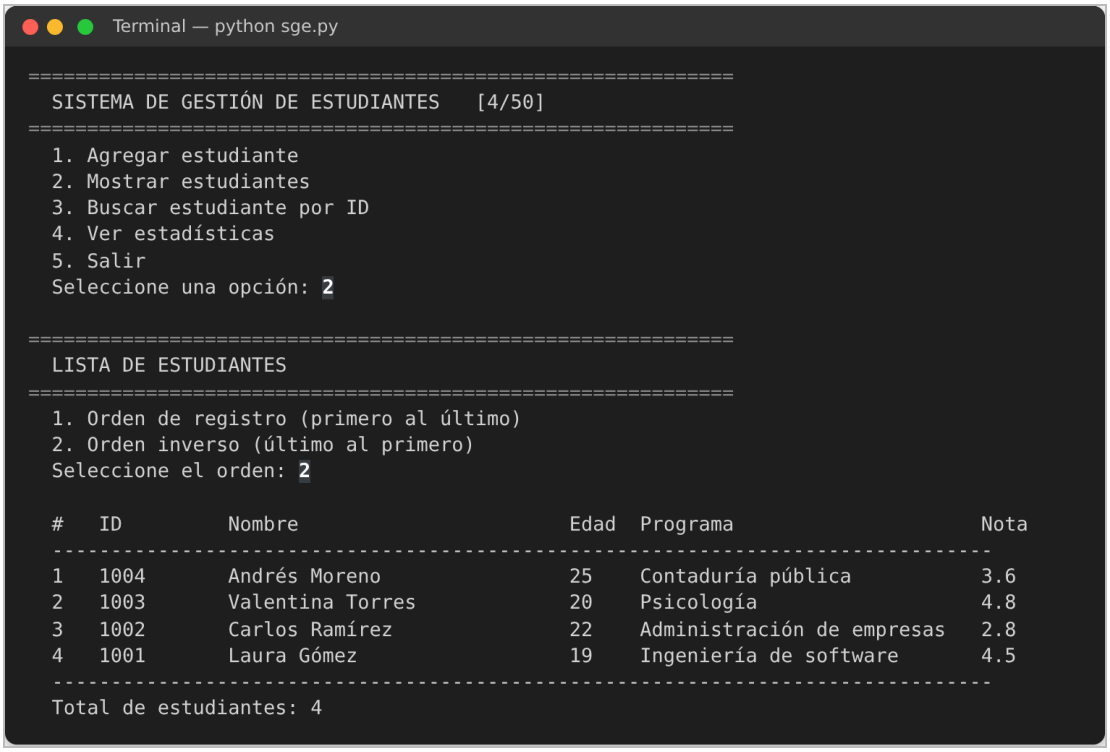


Figura 10

Listado en orden inverso (iteración descendente)



Búsqueda exitosa y fallida

Las Figuras 11 a 13 muestran una búsqueda exitosa en un caso intermedio, una en el mejor caso y una búsqueda fallida, que corresponde al peor caso.

Figura 11

Búsqueda exitosa del ID 1003 con tres comparaciones

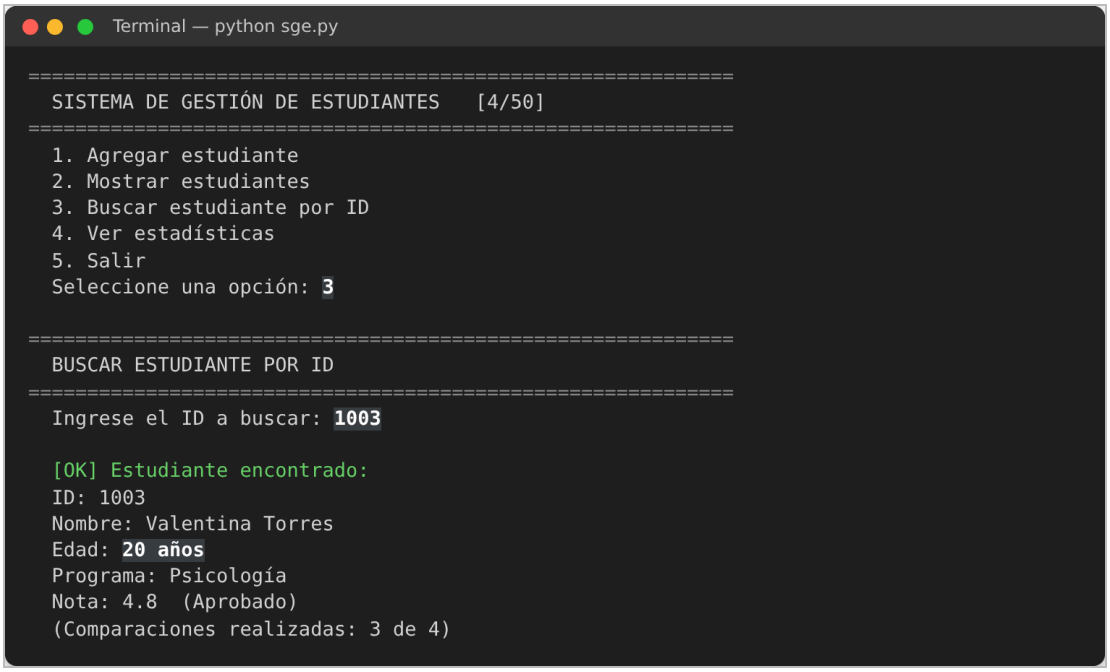


Figura 12

Búsqueda en el mejor caso: ID 1001 con una comparación

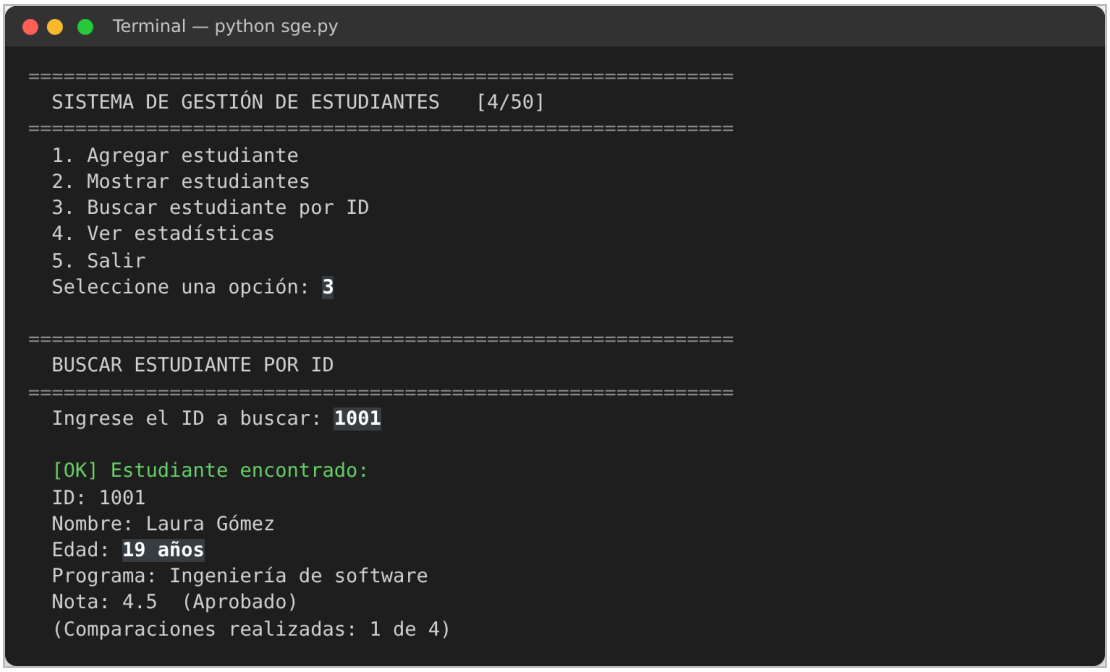


Figura 13

Búsqueda fallida del ID 9999 (peor caso)

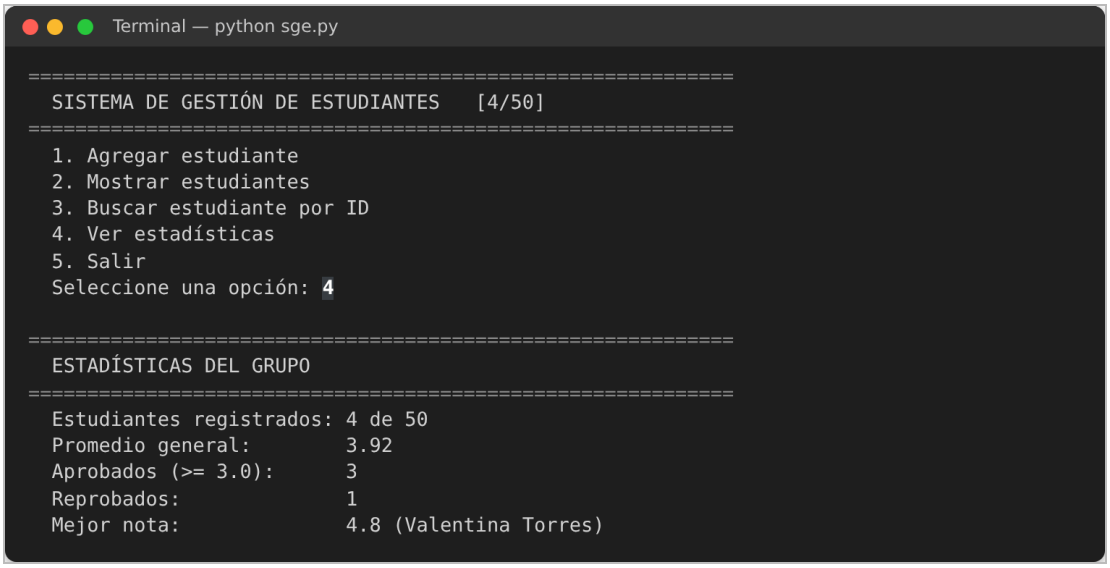


Estadísticas y salida del sistema

La Figura 14 presenta las estadísticas del grupo y la Figura 15 la salida del sistema mediante la opción 5.

Figura 14

Estadísticas del grupo



Nota. Comprobación manual del promedio: (4.5 + 2.8 + 4.8 + 3.6) / 4 = 3.925, que el programa muestra como 3.92.

Figura 15

Salida del sistema al activar la bandera bandera_salir



Situaciones límite

Para probar el límite de capacidad sin registrar 50 estudiantes, ejecuté el programa con `MAX_ESTUDIANTES` reducido temporalmente a 2 (Figura 16). En esa misma prueba registré una nota de 0.0, que es el valor mínimo permitido. La Figura 17 muestra la interrupción del programa en medio de un registro.

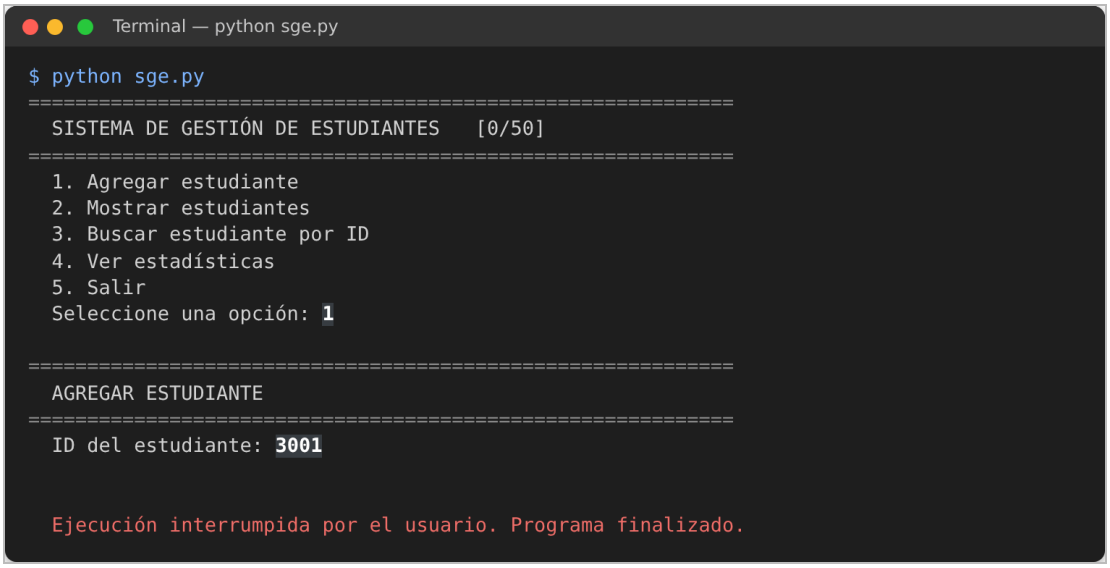
Figura 16

Nota en el límite inferior y rechazo al alcanzar la capacidad máxima



Figura 17

Cierre ordenado ante una interrupción del usuario



Descripción de los casos de prueba

La Tabla 10 resume los datos de entrada, el resultado esperado y el resultado obtenido en cada caso de prueba. Los 17 casos dieron el resultado esperado.

Tabla 10

Casos de prueba ejecutados

ID	Caso	Entrada	Resultado esperado	Resultado obtenido	Fig.
CP-01	Opción no numérica en el menú	abc	Aviso y nueva solicitud	Aviso; el programa no se cierra	5
CP-02	Opción fuera de rango	9	Aviso de rango	“El valor debe ser menor o igual a 5”	5
CP-03	Mostrar y buscar sin datos	2 y 3	Mensaje de lista vacía	“No hay estudiantes registrados”	5, 6
CP-04	ID con letras	doce	Rechazo	Pide nuevamente el ID	7
CP-05	Nombre con números	Laura Gómez 2	Rechazo	“Solo se permiten letras y espacios”	7
CP-06	Nombre con espacios extra y minúsculas	laura gómez	Se normaliza	Se guarda como “Laura Gómez”	7, 9
CP-07	Edad fuera de rango	8	Rechazo (mínimo 14)	“Debe ser mayor o igual a 14”	7
CP-08	Nota mayor a 5.0 y nota con coma	6 y luego 4,5	Rechaza 6, acepta 4,5	Se guarda 4.5	7
CP-09	ID duplicado	1001	Rechazo y nuevo ID	“Ya existe un estudiante con el ID 1001”	8
CP-10	Listado en ambos órdenes	Orden 1 y 2	1001 a 1004 y 1004 a 1001	Correcto	9, 10
CP-11	Búsqueda exitosa	1003 y 1001	Ficha; 3 y 1 comparaciones	Correcto	11, 12
CP-12	Búsqueda fallida (peor caso)	9999	“No existe”; 4 comparaciones	Correcto	13
CP-13	Capacidad máxima	Tope 2; tercer registro	Rechazo sin pedir datos	“Capacidad máxima alcanzada”	16
CP-14	Nota en el límite inferior	0	Aceptada	Registrada como 0.0	16
CP-15	Estadísticas	4.5, 2.8, 4.8 y 3.6	Promedio 3.92; 3 aprobados	Coincide con el cálculo manual	14
CP-16	Interrupción	Ctrl+C durante un registro	Cierre ordenado	Mensaje de cierre, sin errores técnicos	17
CP-17	Salir del sistema	5	Fin del ciclo	Correcto	15

Análisis y reflexión

Evaluación del cumplimiento de objetivos

La Tabla 11 contrasta cada requisito de la actividad con la evidencia que demuestra su cumplimiento.

Tabla 11

Cumplimiento de los requisitos de la actividad

Requisito	Estado	Evidencia
Diagrama de flujo con simbología estándar	Cumplido	Figuras 1 a 4 y Tabla 3
Agregar, mostrar, buscar por ID y salir	Cumplido	Figuras 5 a 15
Uso de `if`, switch (`match-case`), `while` y `for`	Cumplido	Tabla 5
Iteraciones ascendentes y descendentes	Cumplido	Figuras 3, 9 y 10
Contador y bandera (además, acumulador)	Cumplido	Tabla 6
Arreglo o lista con tamaño máximo	Cumplido	Caso CP-13 y Figura 16
Funciones reutilizables	Cumplido	Tabla 9 (13 funciones)
Validación de entradas y manejo de errores	Cumplido	Casos CP-01 a CP-09 y CP-16
Análisis Big-O	Cumplido	Tablas 7 y 8

Conclusiones técnicas

Hacer primero el diagrama y la prueba de escritorio sí ahorró tiempo. La lógica de la búsqueda y del ciclo principal funcionó en el primer intento porque ya la había ejecutado en papel; los errores que aparecieron después fueron de formato y no de lógica.

Elegir bien la estructura de control depende de saber cuántas repeticiones habrá. Cuando no se conoce el número de repeticiones, como en el menú y en las validaciones, corresponde un `while`. Cuando se recorre una colección de tamaño conocido, corresponde un `for`. Las variables especiales también hacen el código más claro: con una bandera, la condición del ciclo se lee casi como una frase (“mientras no se deba salir”).

Por último, la búsqueda lineal es O(n). Para el tamaño de este sistema es suficiente, pero las mediciones muestran que, a gran escala, una estructura con acceso por clave como el diccionario es claramente superior.

Dificultades encontradas y soluciones implementadas

Durante el desarrollo se presentaron varios problemas técnicos. La Tabla 12 los relaciona con la solución que apliqué en cada caso.

Tabla 12

Dificultades encontradas y soluciones aplicadas

Dificultad	Solución
Python no tiene `switch` como Java o C.	Usé `match-case` (Python 3.10 o superior) y dejé documentado en el README que se requiere esa versión.
Python no tiene `do-while`, y para validar un dato hay que pedirlo al menos una vez.	Simulé el `do-while` con `while True` y un `return` cuando el dato es válido.
Si el usuario escribía letras en un campo numérico, el programa se cerraba con `ValueError`.	Encerré la conversión en `try` y `except` dentro de funciones de lectura reutilizables.
En Colombia es común escribir las notas con coma (4,5), y `float("4,5")` falla.	Reemplacé la coma por punto antes de convertir.
En la primera prueba, el programa académico aparecía como “Ingeniería De Software”, porque `title()` pone en mayúscula cada palabra.	Usé `title()` solo para nombres propios y, para el programa, solo la primera letra en mayúscula.
Los programas con nombre largo descuadraban las columnas del listado.	Amplié la columna a 28 caracteres y recorté el texto si es más largo.
Probar el límite de 50 estudiantes a mano era muy lento.	Como el límite es una constante, lo reduje temporalmente a 2. Esto mostró la ventaja de no escribir valores fijos dentro del código.

Aprendizajes significativos

El aprendizaje más importante fue entender que un programa que funciona no es lo mismo que un programa robusto: la mayor parte del código del SGE no está en las operaciones principales, sino en validar lo que escribe el usuario. También comprendí mejor la notación Big-O. Ver que el contador de comparaciones daba 1, 3 y 4 según el caso hizo concreta la diferencia entre el mejor y el peor caso, y la medición de tiempos mostró por qué la estructura de datos importa cuando los datos crecen. Por último, dividir el programa en funciones pequeñas me permitió reutilizar la misma búsqueda para dos propósitos y encontrar los errores más rápido.

Reflexión sobre las estructuras de control

Para cada estructura evalué al menos una alternativa antes de decidirme. La Tabla 13 presenta ese análisis crítico y las razones por las que se descartaron las alternativas.

Tabla 13

Alternativas consideradas para las estructuras de control

Decisión tomada	Alternativa considerada	Por qué se descartó
`while not bandera_salir` para el menú	`while True` con `break` en la opción 5	Funciona igual, pero la condición de salida queda escondida dentro del `match`. Con la bandera, la regla de terminación se lee en la primera línea del ciclo.
`match-case`	Cadena de `if` y `elif`	Con cinco opciones, `match` es más legible y representa directamente el rombo de decisión múltiple. El `if` sí lo usé donde las condiciones son rangos, porque `match` no es natural para eso.
`for` con `break` en la búsqueda	`while posicion < n and not encontrado`	Ambas son correctas. Preferí el `for` porque el número máximo de vueltas es conocido y así se evita manejar el incremento a mano, una causa común de ciclos infinitos.
Un solo `for` con paso variable para los dos órdenes	Dos ciclos distintos, o `lista[::-1]`	Dos ciclos duplicarían código, y `lista[::-1]` crea una copia invertida con O(n) de memoria extra.
Validación con ciclos	Validación recursiva	Cada error dejaría una llamada abierta en la pila y, con muchos errores seguidos, podría alcanzarse el límite de recursión. El ciclo usa memoria constante.
Lista con tope de 50	Lista sin límite o diccionario	El tope permite practicar el control de un arreglo fijo; el diccionario quedó como mejora futura para volúmenes grandes.

Mejoras futuras

Si continuara el proyecto, agregaría cuatro mejoras: guardar los datos en un archivo CSV o JSON para que no se pierdan al cerrar el programa, usar un diccionario como índice por ID para que la búsqueda sea O(1), incluir opciones para modificar y eliminar estudiantes, y escribir pruebas automáticas con `unittest`.

Referencias

Bhargava, A. Y. (2016). *Grokking algorithms: An illustrated guide for programmers and other curious people*. Manning Publications.

Bucher, B. y van Rossum, G. (2020). *PEP 634 – Structural pattern matching: Specification*. Python Software Foundation. https://peps.python.org/pep-0634/

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

International Organization for Standardization. (1985). *Information processing — Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts* (ISO Standard No. 5807:1985). https://www.iso.org/standard/11955.html

Joyanes Aguilar, L. (2008). *Fundamentos de programación: Algoritmos, estructuras de datos y objetos* (4.ª ed.). McGraw-Hill.

Mermaid. (s. f.). *Flowcharts – Basic syntax*. Recuperado el 2 de noviembre de 2026, de https://mermaid.js.org/syntax/flowchart.html

Python Software Foundation. (s. f.-a). *Errors and exceptions*. Recuperado el 2 de noviembre de 2026, de https://docs.python.org/3/tutorial/errors.html

Python Software Foundation. (s. f.-b). *More control flow tools*. Recuperado el 2 de noviembre de 2026, de https://docs.python.org/3/tutorial/controlflow.html

Python Software Foundation. (s. f.-c). *TimeComplexity*. Recuperado el 2 de noviembre de 2026, de https://wiki.python.org/moin/TimeComplexity

Apéndice A

Código fuente completo del programa sge.py

```
"""
Sistema de Gestión de Estudiantes (SGE) básico
-----------------------------------------------
Aplicación por consola desarrollada en Python 3 para la asignatura
Algoritmos y Programación (grupo 2310 - 1A), Fundación Universitaria Compensar.

Autor: Joan Sebastián Quiñones Arias
Docente: Eliana Marcela Paez Perdomo
Repositorio: https://github.com/Sebastian20031221/Sistema_de_gestion_de_estudiantes-SGE-
Funcionalidades:
    1. Agregar estudiante
    2. Mostrar estudiantes (orden ascendente o descendente)
    3. Buscar estudiante por ID
    4. Ver estadísticas del grupo
    5. Salir del sistema

Requiere Python 3.10 o superior (usa la sentencia match-case).
"""

# ---------------------------------------------------------------------------
# CONSTANTES DE CONFIGURACIÓN
# ---------------------------------------------------------------------------
# Aunque las listas de Python crecen solas, se fija un tope para simular el
# comportamiento de un arreglo de tamaño fijo y practicar el control de límites
# que pide la actividad. Cambiar este valor no exige tocar el resto del código.
MAX_ESTUDIANTES = 50

NOTA_MINIMA = 0.0
NOTA_MAXIMA = 5.0          # escala de calificación usada en Colombia
NOTA_APROBATORIA = 3.0
EDAD_MINIMA = 14
EDAD_MAXIMA = 100


# ---------------------------------------------------------------------------
# FUNCIONES DE LECTURA Y VALIDACIÓN DE ENTRADAS
# ---------------------------------------------------------------------------
# Se separan en funciones propias porque la misma validación se necesita en
# varios lugares (menú, agregar, buscar). Así se evita repetir código y el
# programa nunca se cae si el usuario escribe algo inesperado.

def leer_entero(mensaje, minimo=None, maximo=None):
    """Pide un número entero hasta que el usuario ingrese uno válido."""
    while True:  # se repite hasta obtener un dato correcto
        texto = input(mensaje).strip()
        try:
            valor = int(texto)
        except ValueError:
            # int("abc") lanza ValueError: lo atrapamos para no detener el programa
            print("  [!] Debe ingresar un número entero. Intente de nuevo.")
            continue
        if minimo is not None and valor < minimo:
            print(f"  [!] El valor debe ser mayor o igual a {minimo}.")
        elif maximo is not None and valor > maximo:
```

```
            print(f"  [!] El valor debe ser menor o igual a {maximo}.")
        else:
            return valor


def leer_decimal(mensaje, minimo, maximo):
    """Pide un número decimal dentro de un rango (acepta coma o punto)."""
    while True:
        # En Colombia es común escribir 4,5 en lugar de 4.5, por eso se reemplaza
        texto = input(mensaje).strip().replace(",", ".")
        try:
            valor = float(texto)
        except ValueError:
            print("  [!] Debe ingresar un número (ejemplo: 4.5). Intente de nuevo.")
            continue
        if minimo <= valor <= maximo:
            return round(valor, 1)
        print(f"  [!] El valor debe estar entre {minimo} y {maximo}.")


def leer_texto(mensaje, solo_letras=False, es_nombre_propio=False):
    """Pide un texto no vacío; opcionalmente exige que tenga solo letras y
espacios."""
    while True:
        texto = " ".join(input(mensaje).split())  # quita espacios sobrantes
        if texto == "":
            print("  [!] Este campo no puede quedar vacío.")
        elif solo_letras and not texto.replace(" ", "").isalpha():
            print("  [!] Solo se permiten letras y espacios.")
        else:
            if es_nombre_propio:
                return texto.title()          # "laura gómez" -> "Laura Gómez"
            return texto[0].upper() + texto[1:]  # "ingeniería de software" ->
"Ingeniería de software"


# ---------------------------------------------------------------------------
# FUNCIONES DE BÚSQUEDA Y PRESENTACIÓN (UNIDADES REUTILIZABLES)
# ---------------------------------------------------------------------------

def buscar_indice_por_id(estudiantes, id_buscado):
    """
    Búsqueda lineal (secuencial) de un estudiante por su ID.

    Retorna una tupla (indice, comparaciones):
      - indice: posición del estudiante en la lista, o -1 si no existe.
      - comparaciones: cuántas veces se comparó, útil para analizar Big-O.

    Se usa una variable bandera 'encontrado' y se corta el ciclo con break en
    cuanto aparece el ID, de modo que el mejor caso es O(1) y el peor O(n).
    """
    encontrado = False      # bandera: indica si ya apareció el ID
    indice = -1
    comparaciones = 0       # contador de comparaciones realizadas

    for posicion in range(len(estudiantes)):   # iteración ascendente
        comparaciones += 1
        if estudiantes[posicion]["id"] == id_buscado:
            encontrado = True
            indice = posicion
```

```
113             break           # no tiene sentido seguir recorriendo
114
115     # La bandera deja explícito el resultado: si nunca se activó, se retorna -1
116     return (indice if encontrado else -1), comparaciones
117
118
119 def mostrar_estudiante(estudiante):
120     """Imprime los datos de UN estudiante. Se reutiliza en Mostrar y en Buscar."""
121     estado = "Aprobado" if estudiante["nota"] >= NOTA_APROBATORIA else "Reprobado"
122     print(f"  ID: {estudiante['id']}")
123     print(f"  Nombre: {estudiante['nombre']}")
124     print(f"  Edad: {estudiante['edad']} años")
125     print(f"  Programa: {estudiante['programa']}")
126     print(f"  Nota: {estudiante['nota']:.1f}  ({estado})")
127
128
129 def imprimir_fila(numero, estudiante):
130     """Imprime un estudiante como fila de tabla (formato compacto para listados)."""
131     print(f"  {numero:<3} {estudiante['id']:<10} {estudiante['nombre']:<28} "
132           f"{estudiante['edad']:<5} {estudiante['programa'][:28]:<28}
{estudiante['nota']:.1f}")
133
134
135 def imprimir_titulo(texto):
136     """Encabezado uniforme para cada sección de la consola."""
137     print("\n" + "=" * 60)
138     print(f"  {texto}")
139     print("=" * 60)
140
141
142 # ------------------------------------------------------------------------
143 # FUNCIONALIDADES DEL MENÚ
144 # ------------------------------------------------------------------------
145
146 def agregar_estudiante(estudiantes, total_estudiantes):
147     """
148     Registra un nuevo estudiante en la lista.
149     Recibe el contador actual y retorna el contador actualizado.
150     """
151     imprimir_titulo("AGREGAR ESTUDIANTE")
152
153     # Decisión simple: controlar que no se supere el tamaño máximo
154     if total_estudiantes >= MAX_ESTUDIANTES:
155         print(f"  [!] Capacidad máxima alcanzada ({MAX_ESTUDIANTES} estudiantes).")
156         return total_estudiantes
157
158     # El ID debe ser único; se repite la lectura mientras ya exista
159     id_estudiante = leer_entero("  ID del estudiante: ", minimo=1)
160     indice, _ = buscar_indice_por_id(estudiantes, id_estudiante)
161     while indice != -1:
162         print(f"  [!] Ya existe un estudiante con el ID {id_estudiante}.")
163         id_estudiante = leer_entero("  Ingrese otro ID: ", minimo=1)
164         indice, _ = buscar_indice_por_id(estudiantes, id_estudiante)
165
166     nombre = leer_texto("  Nombre completo: ", solo_letras=True,
es_nombre_propio=True)
167     edad = leer_entero("  Edad: ", minimo=EDAD_MINIMA, maximo=EDAD_MAXIMA)
168     programa = leer_texto("  Programa académico: ")
169     nota = leer_decimal("  Nota promedio (0.0 - 5.0): ", NOTA_MINIMA, NOTA_MAXIMA)
170
```

```
171     # Cada estudiante es un diccionario: agrupa sus datos con nombres claros
172     nuevo_estudiante = {
173         "id": id_estudiante,
174         "nombre": nombre,
175         "edad": edad,
176         "programa": programa,
177         "nota": nota,
178     }
179     estudiantes.append(nuevo_estudiante)   # O(1) amortizado
180     total_estudiantes += 1                 # variable de conteo
181 
182     print(f"\n  [OK] Estudiante registrado. Total actual: {total_estudiantes}")
183     return total_estudiantes
184 
185 
186 def mostrar_estudiantes(estudiantes, total_estudiantes):
187     """Lista todos los estudiantes en orden de registro ascendente o descendente."""
188     imprimir_titulo("LISTA DE ESTUDIANTES")
189 
190     if total_estudiantes == 0:
191         print("  No hay estudiantes registrados todavía.")
192         return
193 
194     print("  1. Orden de registro (primero al último)")
195     print("  2. Orden inverso (último al primero)")
196     orden = leer_entero("  Seleccione el orden: ", minimo=1, maximo=2)
197 
198     if orden == 1:
199         # Iteración ascendente: 0, 1, 2, ..., n-1
200         inicio, fin, paso = 0, total_estudiantes, 1
201     else:
202         # Iteración descendente: n-1, n-2, ..., 0
203         inicio, fin, paso = total_estudiantes - 1, -1, -1
204 
205     print(f"\n  {'#':<3} {'ID':<10} {'Nombre':<28} {'Edad':<5} {'Programa':<28} Nota")
206     print("  " + "-" * 80)
207     numero = 1
208     for posicion in range(inicio, fin, paso):
209         imprimir_fila(numero, estudiantes[posicion])
210         numero += 1
211     print("  " + "-" * 80)
212     print(f"  Total de estudiantes: {total_estudiantes}")
213 
214 
215 def buscar_estudiante(estudiantes, total_estudiantes):
216     """Busca un estudiante por ID e informa si fue encontrado o no."""
217     imprimir_titulo("BUSCAR ESTUDIANTE POR ID")
218 
219     if total_estudiantes == 0:
220         print("  No hay estudiantes registrados para buscar.")
221         return
222 
223     id_buscado = leer_entero("  Ingrese el ID a buscar: ", minimo=1)
224     indice, comparaciones = buscar_indice_por_id(estudiantes, id_buscado)
225 
226     # Decisión simple: ¿se encontró o no?
227     if indice != -1:
228         print("\n  [OK] Estudiante encontrado:")
229         mostrar_estudiante(estudiantes[indice])
230     else:
```

```
231         print(f"\n  [X] No existe ningún estudiante con el ID {id_buscado}.")
232     print(f"  (Comparaciones realizadas: {comparaciones} de {total_estudiantes})")
233 
234 
235 def mostrar_estadisticas(estudiantes, total_estudiantes):
236     """Calcula promedio, mejor nota y aprobados recorriendo la lista una sola vez."""
237     imprimir_titulo("ESTADÍSTICAS DEL GRUPO")
238 
239     if total_estudiantes == 0:
240         print("  No hay datos para calcular estadísticas.")
241         return
242 
243     suma_notas = 0.0                # acumulador
244     aprobados = 0                   # contador
245     mejor = estudiantes[0]          # variable auxiliar (máximo)
246 
247     # Un solo recorrido O(n) calcula las tres cosas a la vez
248     for estudiante in estudiantes:
249         suma_notas += estudiante["nota"]
250         if estudiante["nota"] >= NOTA_APROBATORIA:
251             aprobados += 1
252         if estudiante["nota"] > mejor["nota"]:
253             mejor = estudiante
254 
255     promedio = suma_notas / total_estudiantes
256     print(f"  Estudiantes registrados: {total_estudiantes} de {MAX_ESTUDIANTES}")
257     print(f"  Promedio general:        {promedio:.2f}")
258     print(f"  Aprobados (>= {NOTA_APROBATORIA}):      {aprobados}")
259     print(f"  Reprobados:              {total_estudiantes - aprobados}")
260     print(f"  Mejor nota:              {mejor['nota']:.1f} ({mejor['nombre']})")
261 
262 
263 def mostrar_menu(total_estudiantes):
264     """Imprime el menú principal (salida)."""
265     imprimir_titulo(f"SISTEMA DE GESTIÓN DE ESTUDIANTES
[{total_estudiantes}/{MAX_ESTUDIANTES}]")
266     print("  1. Agregar estudiante")
267     print("  2. Mostrar estudiantes")
268     print("  3. Buscar estudiante por ID")
269     print("  4. Ver estadísticas")
270     print("  5. Salir")
271 
272 
273 # ---------------------------------------------------------------------------
274 # PROGRAMA PRINCIPAL
275 # ---------------------------------------------------------------------------
276 
277 def main():
278     estudiantes = []          # estructura de datos: lista de diccionarios
279     total_estudiantes = 0     # variable de conteo
280     bandera_salir = False     # bandera que controla el ciclo principal
281 
282     # Repetición condicionada al comienzo: el menú se muestra mientras
283     # el usuario no haya elegido salir.
284     while not bandera_salir:
285         mostrar_menu(total_estudiantes)
286         opcion = leer_entero("  Seleccione una opción: ", minimo=1, maximo=5)
287 
288         # Decisión múltiple: equivalente en Python al switch de otros lenguajes
289         match opcion:
```

```
290             case 1:
291                 total_estudiantes = agregar_estudiante(estudiantes, total_estudiantes)
292             case 2:
293                 mostrar_estudiantes(estudiantes, total_estudiantes)
294             case 3:
295                 buscar_estudiante(estudiantes, total_estudiantes)
296             case 4:
297                 mostrar_estadisticas(estudiantes, total_estudiantes)
298             case 5:
299                 bandera_salir = True   # se activa la bandera y el while termina
300 
301     print("\n  Gracias por usar el SGE. ¡Hasta pronto!\n")
302 
303 
304 if __name__ == "__main__":
305     try:
306         main()
307     except (KeyboardInterrupt, EOFError):
308         # Ctrl+C o fin de entrada: se cierra de forma ordenada, sin traceback
309         print("\n\n  Ejecución interrumpida por el usuario. Programa finalizado.\n")
```